

Laboratorium 10. Zarządzanie procesem testowania. Modelowanie testów.

Sviatoslav Protsyk 143540

Zad 1.

1) Na czym polega proces zarządzania testami?

Proces zarządzania testami to całokształt działań związanych z planowaniem, organizacją, monitorowaniem i kontrolą czynności testowych w projekcie. Polega on na ustaleniu strategii testowania, alokacji zasobów (ludzi, środowisk, narzędzi), tworzeniu harmonogramów, śledzeniu postępów, zarządzaniu zgłoszonymi defektami oraz raportowaniu jakości oprogramowania do interesariuszy, aby na tej podstawie mogli oni podejmować decyzje biznesowe.

2) Jakie są cele procesu testowania?

Główne cele testowania (zgodnie ze standardami np. ISTQB) to:

- **Wykrywanie defektów:** Znalezienie jak największej liczby błędów przed wdrożeniem systemu na produkcję.
- **Budowanie zaufania do jakości:** Potwierdzenie, że system działa zgodnie z wymaganiami i spełnia oczekiwania użytkowników.
- **Zapobieganie defektom:** Wczesne angażowanie testerów (np. przeglądy wymagań) pozwala uniknąć wprowadzania błędów do kodu.
- **Dostarczanie informacji:** Dostarczenie interesariuszom rzetelnej informacji o aktualnym stanie i jakości oprogramowania w celu podjęcia decyzji o wydaniu.
- **Weryfikacja i walidacja:** Sprawdzenie, czy zbudowano system poprawnie (weryfikacja) oraz czy zbudowano właściwy system dla użytkownika (walidacja).

3) Wymienić rodzaje testów.

Testy można dzielić na wiele sposobów, ale najczęstszy, standardowy podział obejmuje:

- **Poziomy testów:** * Testy jednostkowe (sprawdzanie pojedynczych funkcji/modułów).
 - Testy integracyjne (sprawdzanie współpracy między modułami/systemami).

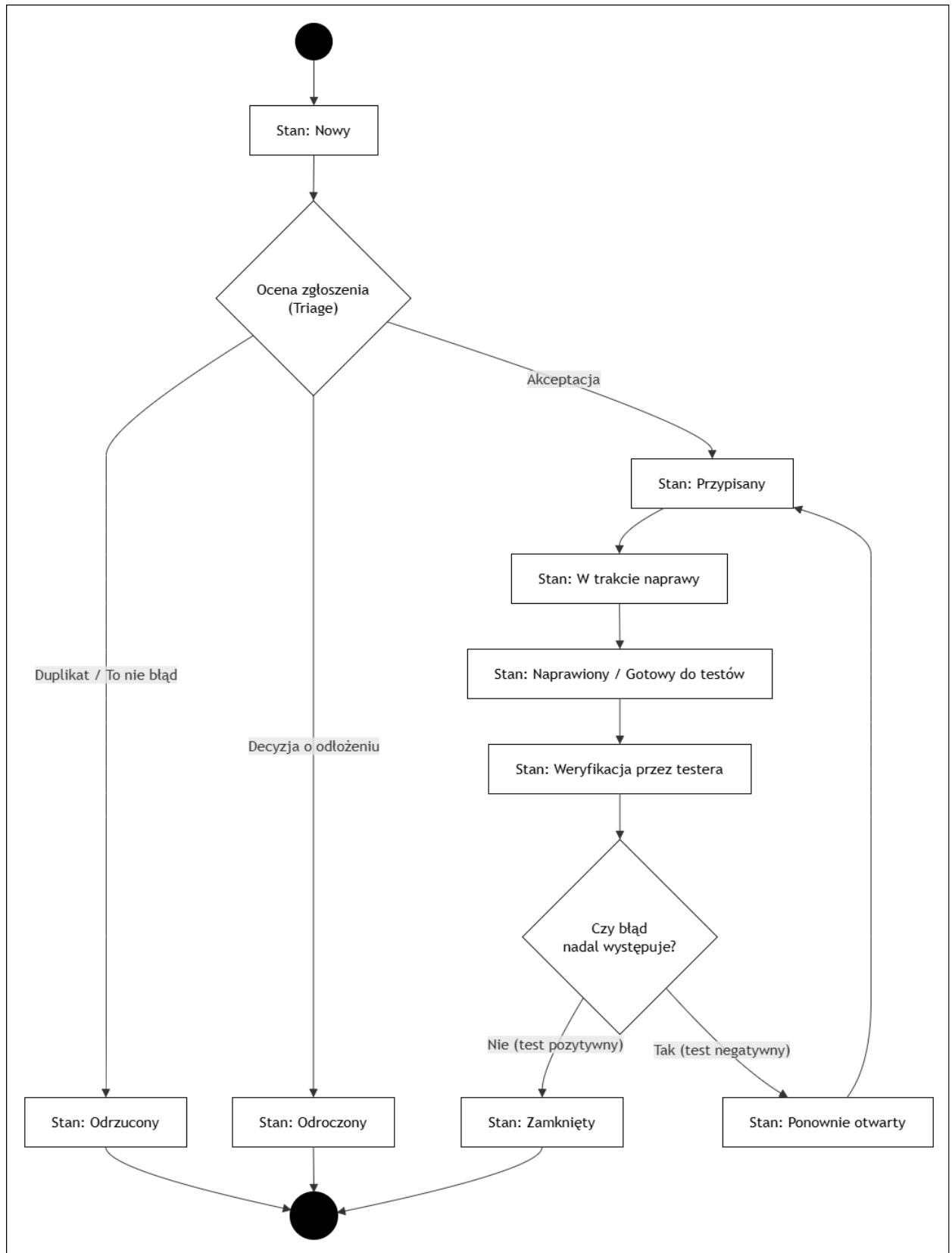
- Testy systemowe (sprawdzanie całego, zintegrowanego systemu).
- Testy akceptacyjne (sprawdzanie systemu z perspektywy klienta/użytkownika końcowego).
- **Typy testów:**
 - Funkcjonalne (co system robi).
 - Niefunkcjonalne (jak system działa – np. testy wydajnościowe, bezpieczeństwa, użyteczności).
 - Białoskrzynkowe / strukturalne (testowanie struktury kodu).
 - Związane ze zmianami (testy potwierdzające / re-testy oraz testy regresji).

4) Wymienić elementy planu testów.

Zgodnie ze standardami inżynierii oprogramowania (np. IEEE 829), profesjonalny plan testów powinien zawierać m.in.:

- **Identyfikator i wprowadzenie:** Cel dokumentu i opis projektu.
- **Elementy testowane:** Co dokładnie będzie poddane testom (np. konkretne moduły).
- **Cechy testowane i nietestowane:** Wyraźne określenie zakresu (in/out of scope).
- **Podejście (Strategia testów):** Jak testy będą przeprowadzane, jakie techniki zostaną użyte.
- **Kryteria wejścia i wyjścia:** Kiedy można rozpocząć testy i jakie warunki muszą być spełnione, aby uznać je za zakończone.
- **Środowisko testowe:** Wymagania sprzętowe, oprogramowanie, dane testowe.
- **Zadania i harmonogram:** Kto, co i kiedy będzie robił.
- **Role i odpowiedzialności:** Przydział zadań członkom zespołu.
- **Ryzyka i przypadki awaryjne:** Potencjalne problemy i plany zapobiegawcze.

2. Opis procesu: Cykl życia błędu (Bug Life Cycle)



Powyższy diagram aktywności przedstawia przepływ stanów, przez które przechodzi zgłoszony błąd (defekt) od momentu jego zarejestrowania aż do ostatecznego zakończenia prac. Proces składa się z następujących etapów:

- **Stan: Nowy**
 - *Opis:* Błąd został zidentyfikowany (np. przez testera) i dodany do systemu śledzenia błędów.
 - *Warunek wyjścia:* Rozpoczęcie oceny zgłoszenia (Triage).
- **Decyzja: Ocena zgłoszenia (Triage)** Na tym etapie zespół lub kierownik decyduje, co zrobić ze zgłoszeniem:
 - *Akceptacja:* Błąd jest rzeczywisty i ważny -> przechodzi do stanu **Przypisany**.
 - *Duplikat / To nie błąd:* Zgłoszenie jest bezzasadne lub już istnieje -> przechodzi do stanu **Odrzucony** (koniec procesu).
 - *Decyzja o odłożeniu:* Błąd istnieje, ale jego naprawa ma niski priorytet -> przechodzi do stanu **Odroczony** (koniec procesu na ten moment).
- **Stan: Przypisany**
 - *Opis:* Błąd został zaakceptowany i przydzielony do konkretnego programisty, który ma go naprawić.
 - *Warunek wyjścia:* Programista rozpoczyna pracę, zmieniając stan na **W trakcie naprawy**.
- **Stan: W trakcie naprawy**
 - *Opis:* Trwają aktywne prace programistyczne nad usunięciem defektu w kodzie.
 - *Warunek wyjścia:* Kod zostaje poprawiony, a błąd przechodzi w stan **Naprawiony / Gotowy do testów**.
- **Stan: Naprawiony / Gotowy do testów**
 - *Opis:* Poprawka została wgrana na środowisko testowe i oczekuje na weryfikację.
 - *Warunek wyjścia:* Tester przejmuje zadanie, zmieniając stan na **Weryfikacja przez testera**.
- **Stan: Weryfikacja przez testera (Re-test)**
 - *Opis:* Tester wykonuje testy potwierdzające, aby upewnić się, że wdrożona poprawka faktycznie usunęła błąd.
- **Decyzja: Czy błąd nadal występuje?** Na podstawie wyników weryfikacji tester podejmuje decyzję:

- *Nie (test pozytywny)*: Poprawka działa poprawnie -> błąd przechodzi do stanu **Zamknięty** (ostateczny koniec procesu).
- *Tak (test negatywny)*: Poprawka nie zadziałała lub zepsuła coś innego - > błąd przechodzi do stanu **Ponownie otwarty**.

- **Stan: Ponownie otwarty**

- *Opis*: Informacja dla zespołu, że podjęta próba naprawy zakończyła się niepowodzeniem.
- *Warunek wyjścia*: Błąd natychmiast wraca do stanu **Przypisany** i programista musi ponownie rozpocząć proces naprawy (pętla zwrotna).